

GazeComm

Project Title : AI-Powered Eyegaze and Blink Control: A Hands-Free Interface for Accessibility and Text-to-Speech.

Project Aim :

To design an AI-driven human–computer interaction system that enables seamless mouse control through eye gaze and gesture-based actions (click, hold) using eye blinks. The system will also provide real-time text-to-speech feedback, enabling users to have any highlighted text done by eye-controlled mouse, read aloud instantly.

Objective :

- Develop a robust and accurate eyegaze tracking system to achieve precise and smooth control of the mouse pointer across the screen.
- Implement a real-time eyeblink detection model to translate different blink patterns (e.g., single, double) into mouse actions like clicking, double-clicking, and holding.
- Integrate the eye-controlled mouse with a text-highlighting and text-to-speech module to read selected on-screen text aloud, enhancing accessibility for users with limited motor skills.
- Evaluate the system's usability, efficiency, and accuracy through rigorous testing with target users to ensure a practical and reliable assistive tool.

Technical Approach :

This project will employ a **Computer Vision** and **Deep Learning** pipeline for real-time, hands-free computer interaction, using a modular approach based on pre-trained Intel OpenVINO models.

Part 1: Eye-Controlled Mouse Movement and Click Mechanism

The core system for controlling the mouse pointer via eyegaze relies on a cascading computer vision pipeline composed of four specialized Deep Learning models. This structure ensures accurate and robust detection of eye movement under varying conditions.

AI/ML Methods, Datasets, Tools, and Frameworks

- **Models:** The detection pipeline employs four Intel Pre-trained Models optimized for the OpenVINO™ framework, ensuring high inference speed suitable for real-time applications:

- **Face Detection:** A CNN for detecting human faces in the input image/video stream.
- **Facial Landmarks:** A regression model for identifying key facial points, including the corners of the eyes, nose, and mouth.
- **Head Pose Estimation:** A model that estimates the yaw, pitch, and roll angles of the user's head.
- **Gaze Estimation:** The final model that uses the cropped eyes, facial landmarks, and head pose angles to accurately determine the 3D gaze vector.
- **Frameworks/Tools:**
 - **OpenVINO™ Toolkit:** Used for model optimization and high-performance inference on Intel hardware.
 - **Python/OpenCV:** Used for handling camera input, image preprocessing, and integrating the mouse control logic.

Design Stages (Gaze-to-Movement Flow)

1. **Input Image Acquisition:** The system continuously captures frames from a webcam.
2. **Face Detection:** The face detection model identifies the user's face, providing a Cropped Face bounding box.
3. **Parallel Processing (Landmarks & Head Pose):**
 - The Landmark Detection Model processes the Cropped Face to detect facial features, specifically isolating and cropping the Left and Right Eye regions.
 - The Head Pose Estimation Model processes the Cropped Face to output the Head Pose Angles.
4. **Gaze Estimation:** The Gaze Estimation Model integrates the Cropped Eyes and Head Pose Angles to produce a precise 3D gaze vector.
5. **Mouse Control (Movement & Clicks):**
 - The gaze vector is mapped to screen coordinates to control the Mouse Pointer Movement Direction.
 - The system uses the Eye Aspect Ratio (EAR), calculated from the facial landmarks, to detect Eyeblinks. Distinct blink patterns (e.g., single short blink) are translated into mouse actions (click, double-click, hold), completing the hands-free control mechanism.

Part 2: Integrating Text-to-Speech (TTS) Functionality

The final step is to integrate the eye-controlled mouse with an accessibility feature that reads highlighted text aloud.

1. **Screen Interaction and Text Highlighting:**
 - The user will use the eyegaze movement to position the cursor at the start of the desired text segment.
 - A defined eyeblink pattern (e.g., a sustained "click-and-hold" blink) will initiate the virtual selection/highlighting of the text as the user moves their gaze across the screen.

2. Text Capture Logic:

- The application will monitor the screen content within the selection bounding box created by the gaze-controlled highlight action.
- An Optical Character Recognition (OCR) tool (or a system-level screen capture/text retrieval API) will be employed to capture the text string from the highlighted area.

3. Speech Generation:

- The captured text string will be passed to a Text-to-Speech (TTS) Engine.
- The TTS engine will synthesize the corresponding speech, which is then played through the computer's speakers, effectively reading the highlighted text aloud.

4. Usability Design: Implement clear visual and auditory feedback (e.g., a momentary cursor change or a sound prompt) to inform the user when the highlight-and-read action has been successfully initiated and completed.

Expected Deliverables :

- Working prototype of eye-controlled mouse system.
- Integrated feature to read highlighted text aloud.
- Design documentation and user testing report.
- Demo video showcasing interactive functionalities.
- Source code and implementation guide.

Intellectual Property / Research Dissemination Plan :

At this stage I plan both tracks in parallel: file a provisional patent for the core technical innovations to secure IP, while preparing a peer-review manuscript for India HCI / related venues to disseminate research insights and usability findings. This dual approach preserves commercial potential while ensuring academic visibility and community impact.

Timeline :

Week	Phase	Description
Week 1–2	Ideation & Exploration	Conceptualize project scope, review literature, and explore existing eye tracking & TTS tools. Experiment with very basic eye gaze and blink detection models.

Week 3–4	Revision Phase	Final tuning of eye gaze tracking model; improve pointer control stability and responsiveness.
Week 5	Blink Action Module	Develop and integrate blink detection for mouse actions: click, hold, drag.
Week 6	Text-to-Speech Integration	Integrate real-time TTS to read highlighted text aloud.
Week 7	System Integration	Merge eye gaze, blink controls, and TTS into a unified GUI-based prototype.
Week 8	Testing & Documentation	Conduct user testing, evaluate performance, and prepare final project report and demo video.